

PRAKTIKUM SISTEM OPERASI

Modul 5: Eksplorasi Proses Xinu



Oleh:

Reza Afiansyah Wibowo

2311104062

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

DIREKTORAT KAMPUS PURWOKERTO

UNIVERSITAS TELKOM

2026

JURNAL

Soal 1 [50 Poin] — Tiga Proses Bergiliran (kwak, kwik, kwek)

Tiga proses (P1, P2, P3) harus mencetak kata secara bergiliran dan berulang membentuk siklus: kwak → kwik → kwek → kwak → ... Pola ini diimplementasikan dengan rantai 3 semaphore: setiap proses menunggu (wait) semaphore miliknya, mencetak kata, lalu memberi sinyal (signal) ke semaphore proses berikutnya dalam siklus. Hanya sem1 yang diinisialisasi dengan nilai 1 (agar P1 berjalan lebih dulu), sedangkan sem2 dan sem3 diinisialisasi 0 sehingga P2 dan P3 menunggu sampai mendapat sinyal.

Source code (main_kwak.c)

/* Nama: Reza Afiansyah Wibowo - NIM: 2311104062 */

```
#include <xinu.h>
```

```
sid32 sem1, sem2, sem3;
```

```
process P1(void);
```

```
process P2(void);
```

```
process P3(void);
```

```
process main(void)
```

```
{
```

```
    sem1 = screate(1); /* P1 jalan duluan */
```

```
    sem2 = screate(0);
```

```
    sem3 = screate(0);
```

```
    resume(create(P1, 1024, 20, "P1", 0));
```

```
    resume(create(P2, 1024, 20, "P2", 0));
```

```
    resume(create(P3, 1024, 20, "P3", 0));
```

```
    return OK;
```

```
}
```

```
process P1(void)
```

```
{
```

```
    while (1) {
```

```
        wait(sem1);
```

```
        printf("kwak\n");
```

```
        signal(sem2);
```

```
    }
```

```

}

process P2(void)
{
    while (1) {
        wait(sem2);
        printf("kwik\n");
        signal(sem3);
    }
}

```

```

process P3(void)
{
    while (1) {
        wait(sem3);
        printf("kwek\n");
        signal(sem1);
    }
}

```

Cara kerja: P1 langsung lolos dari wait(sem1) karena nilai awalnya 1, mencetak "kwak", lalu signal(sem2) membuka jalan P2. P2 mencetak "kwik" lalu signal(sem3) membuka jalan P3. P3 mencetak "kwek" lalu signal(sem1) mengembalikan giliran ke P1, sehingga siklus berulang terus secara konsisten (kwak, kwik, kwek, kwak, kwik, kwek, ...) tanpa tumpang tindih antar proses.

\$ make clean

\$ make

(jalankan Backend VM)

xinu \$ main_kwak

Soal 2 [50 Poin] — Produser-Konsumer dengan Semaphore

Proses produser menghasilkan angka 1 sampai 1000 satu per satu ke dalam sebuah buffer bersama (ukuran 1 slot), dan proses konsumen mengambil serta menampilkan nilai tersebut. Digunakan 2 semaphore penghitung (empty dan full) untuk mengatur giliran produksi/konsumsi, serta 1 semaphore mutex untuk melindungi akses ke variabel buffer agar tidak terjadi race condition saat ditulis/dibaca.

Source code (main_prodcons.c)

```
/* Nama: Reza Afiansyah Wibowo - NIM: 2311104062 */
```

```
#include <xinu.h>
```

```
int32 buffer;
```

```
sid32 empty, full, mutex;
```

```
process produser(void);
```

```
process konsumer(void);
```

```
process main(void)
```

```
{
```

```
    empty = screate(1); /* buffer kosong, boleh diisi */
```

```
    full = screate(0); /* belum ada item untuk dikonsumsi */
```

```
    mutex = screate(1); /* akses eksklusif ke buffer */
```

```
    resume(create(produser, 1024, 20, "produser", 0));
```

```
    resume(create(konsumer, 1024, 20, "konsumer", 0));
```

```
    return OK;
```

```
}
```

```
process produser(void)
```

```
{
```

```
    int32 i;
```

```
    for (i = 1; i <= 1000; i++) {
```

```
        wait(empty);
```

```
        wait(mutex);
```

```
        buffer = i;
```

```
        printf("Produser memproduksi nilai %d\n", i);
```

```
        signal(mutex);
```

```
        signal(full);
```

```
    }
```

```
    return OK;
```

```
}
```

```
process konsumer(void)
```

```
{
```

```
    int32 i, nilai;
```

```

for (i = 1; i <= 1000; i++) {
    wait(full);
    wait(mutex);
    nilai = buffer;
    signal(mutex);
    printf("Konsumer menampilkan nilai %d\n", nilai);
    signal(empty);
}
return OK;
}

```

Cara kerja: produser hanya bisa menulis ke buffer jika `empty > 0` (slot kosong), lalu menurunkan `empty` dan menaikkan `full` sebagai tanda ada item baru. Konsumer hanya bisa membaca jika `full > 0`, lalu menurunkan `full` dan menaikkan `empty` sebagai tanda slot sudah kosong lagi. Karena buffer hanya berukuran 1, kedua proses otomatis bergantian: produser tidak bisa menulis nilai baru sebelum konsumer membaca nilai sebelumnya, sehingga urutan keluaran selalu 1, 2, 3, ..., 1000 walaupun keduanya berjalan sebagai proses konkuren yang terpisah.

\$ make clean

\$ make

(jalankan Backend VM)

xinu \$ main_prodcons